Microsoft Enterprise AI Telemetry & Raw API Specification

Technical Reference for Direct Platform APIs, Raw Payloads, and Authentication

Executive Overview

This document provides a verified, technical reference on the exact **raw APIs and JSON/CSV payload responses** exposed by Microsoft and GitHub for AI agents and developer subscriptions.

All synthetic estimates and derived dashboard metrics have been removed to focus entirely on:
1. **The 4 Primary Agent Pathways in Microsoft**: Exact endpoints, direct raw responses, and required admin credentials.
2. **Enterprise Fixed-Seat Subscriptions**: GitHub Copilot and Microsoft 365 Copilot raw API schemas.
3. **Practical Implementation Strategy With Current Resources**:
* **Fixed Seats Telemetry**: What we need from Balaji (Admin).
* **Type 3 Custom Autonomous Agents**: Live demonstration via our Azure AI Foundry / Azure OpenAI deployment.
4. **Stakeholder Repository Classification**: File-tree analysis to identify agent architecture.

1. The 4 Microsoft Agent Pathways (APIs & Raw Payloads)

Pathway 1: Declarative / No-Code Agents (Microsoft 365 Copilot)

Configuration-driven assistants scoped to SharePoint, OneDrive, and Teams data defined via `declarativeAgent.json`.

1. Technical Context & Telemetry Scope

- Declarative agents execute entirely inside Microsoft 365 Copilot.
- Telemetry for this pathway is provided via the **Microsoft Graph Usage Reports API**. It reports user-level activity across Microsoft 365 applications for licensed users. It does not output raw conversation transcripts or per-agent prompt text.

2. Exact API Endpoint

```
GET https://graph.microsoft.com/v1.0/copilot/reports/getMicrosoft365CopilotUsageUserDetail(period='D28
```

(Supports periods `'D7'`, `'D28'`, `'D90'`, *and* `'D180'`. *Default output is CSV; append* `$format=application/json` *for JSON).*

3. Exact Raw JSON Response Schema

```
{
  "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#Collection(microsoft.graph.copilotUsag
  "value": [
    {
      "reportRefreshDate": "2026-10-24",
      "userPrincipalName": "employee@company.com",
      "displayName": "Alex Smith",
      "lastActivityDate": "2026-10-24",
      "copilotChatLastActivityDate": "2026-10-24",
      "microsoftTeamsCopilotLastActivityDate": "2026-10-23",
      "wordCopilotLastActivityDate": "2026-10-20",
      "excelCopilotLastActivityDate": null,
      "powerPointCopilotLastActivityDate": "2026-10-18",
      "outlookCopilotLastActivityDate": "2026-10-24",
      "oneNoteCopilotLastActivityDate": null,
      "loopCopilotLastActivityDate": null,
      "copilotAgentLastActivityDate": "2026-10-24"
    }
  ]
}
```

4. Credentials & Permissions

- **Permission**: Microsoft Entra Application permission `Reports.Read.All` (Admin Consent required).
- **Supported Roles**: Global Administrator, AI Administrator, Reports Reader, Teams Administrator, Exchange Administrator.
- **Privacy Note**: If "*Conceal user, group, and site names*" is active in Microsoft 365 Org Settings, `userPrincipalName` and `displayName` return anonymized hash values.

Pathway 2: Low-Code Workflow Agents (Microsoft Copilot Studio)

Autonomous chatbots and business workflow agents built on Power Platform and Microsoft Dataverse.

1. Technical Context & Telemetry Scope

- Copilot Studio conversations are automatically logged into the Microsoft Dataverse table `conversationtranscript`.
- The `content` column stores the full activity stream in JSON text. Session outcomes (`Resolved`, `Escalated`, `Abandon`) and survey responses are embedded inside these JSON activity objects.

2. Exact API Endpoint

```
GET https://<org-env>.api.crm.dynamics.com/api/data/v9.2/conversationtranscripts?$select=conversationt
```

3. Exact Raw JSON Response Schema


```
{
  "@odata.context": "https://<org-env>.api.crm.dynamics.com/api/data/v9.2/$metadata#conversationtransc
  "value": [
    {
      "conversationtranscriptid": "7b8e5c32-1244-ef11-840a-000d3a123456",
      "name": "Conversation with Bot - 2026-10-24T14:30:00Z",
      "conversationstarttime": "2026-10-24T14:30:12Z",
      "_bot_conversationtranscriptid_value": "4f9d2a11-0000-ef11-840a-000d3a987654",
      "schematype": "BotFramework",
      "schemaversion": "1.0",
      "metadata": "{\"BotId\":\"4f9d2a11-0000-ef11-840a-000d3a987654\",\"TenantId\":\"9e123456- ... \",\
      "content": "{\"activities\":[{\"type\":\"message\",\"from\":{\"id\":\"user-hashed-id\",\"role\":
    }
  ]
}
```


4. Credentials & Permissions

- **Security Role**: `Bot Transcript Viewer` or `System Administrator` in the Power Platform environment.
- **Credentials**: Entra ID Application User with Client ID, Client Secret, Tenant ID, and Dataverse Environment URL.
- **Storage Note**: Transcripts over 1 MB are batched across multiple records. Default Dataverse bulk deletion removes records older than 30 days unless configured otherwise.

Pathway 3: Code-First Custom Agents (Azure AI Foundry / OpenAI / App Insights)

Full-stack software agents developed in Python or C# deployed to Azure cloud infrastructure.

1. Technical Context & Telemetry Scope

- Custom agents interact directly with Azure OpenAI model deployments and log execution spans to Azure Application Insights. Engineering has full control over request headers and application telemetry baggage.

2. Exact API Endpoints & Raw Responses

A. Azure OpenAI Chat Completions API (Direct Model Telemetry)

- Returned with every inference completion:

```
{
  "id": "chatcmpl-A8bc9d0123456",
  "object": "chat.completion",
  "created": 1729780800,
  "model": "gpt-4o-2024-11-20",
  "choices": [
    {
      "index": 0,
      "message": { "role": "assistant", "content": "I have created ticket #4812." },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 128,
    "completion_tokens": 85,
    "total_tokens": 213
  }
}
```

B. Azure Application Insights REST API (KQL Query)

```
POST https://api.applicationinsights.io/v1/apps/{appId}/query
Content-Type: application/json

{
  "query": "requests | project timestamp, name, duration, success, resultCode, customDimensions"
}
```

- Raw Response Schema:

```
{
  "tables": [
    {
      "name": "PrimaryResult",
      "columns": [
        { "name": "timestamp", "type": "datetime" },
        { "name": "name", "type": "string" },
        { "name": "duration", "type": "real" },
        { "name": "success", "type": "string" },
        { "name": "resultCode", "type": "string" },
        { "name": "customDimensions", "type": "dynamic" }
      ],
      "rows": [
        [
          "2026-10-24T14:32:00.123Z",
          "POST /api/v1/triage",
          1420.5,
          "True",
          "200",
          { "user_email": "employee@company.com", "task": "Hardware_Triage" }
        ]
      ]
    }
  ]
}
```


C. Azure Cost Management Query API

```
POST https://management.azure.com/subscriptions/{subId}/providers/Microsoft.CostManagement/query?api-v
```

- Raw Response Schema:


```
{
  "id": "subscriptions/00000000-...../providers/Microsoft.CostManagement/query/123",
  "properties": {
    "columns": [
      { "name": "PreTaxCost", "type": "Number" },
      { "name": "ResourceGroup", "type": "String" },
      { "name": "Currency", "type": "String" }
    ],
    "rows": [
      [14.28, "claimsGuru", "USD"]
    ]
  }
}
```

3. Credentials & Permissions

- **Azure OpenAI**: Cognitive Services OpenAI User or API Key.
- **App Insights**: Entra Service Principal with `Monitoring Reader` role.
- **Cost Management**: Entra Service Principal with `Cost Management Reader` role.

Pathway 4: Governed Data Agents (Microsoft Fabric Data Agents)

Specialized analytics agents querying OneLake, Lakehouses, and Power BI semantic models.

1. Technical Context & Telemetry Scope

- Fabric Data Agent monitoring is split across three distinct sources: audit event logs, workspace metadata, and capacity compute consumption.

2. Exact API Endpoints & Raw Responses

A. Power BI / Fabric Activity Events API (Audit Trail)

```
GET https://api.powerbi.com/v1.0/myorg/admin/activityevents?startDateTime='2026-10-24T00:00:00.000Z'&e
```

- Raw JSON Response:


```
{
  "activityEventEntities": [
    {
      "Id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "RecordType": 1,
      "CreationTime": "2026-10-24T15:10:00Z",
      "Operation": "ExecuteDAXQuery",
      "OrganizationId": "9e123456-...",
      "UserType": 0,
      "UserKey": "10032001...",
      "WorkSpaceName": "Enterprise Data Lake",
      "ArtifactName": "Sales_Semantic_Model",
      "CapacityId": "ABC12-34567-DEF89",
      "CapacityName": "Premium_P1_Capacity",
      "UserId": "employee@company.com",
      "IsSuccess": true
    }
  ]
}
```


B. Fabric Get Workspace API (Topology)

```
GET https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}
```

- Raw JSON Response:

```
{
  "id": "cf012345-6789-abcd-ef01-23456789abcd",
  "displayName": "Enterprise Data Lake",
  "description": "OneLake Sales & Operations",
  "type": "Workspace",
  "capacityId": "ABC12-34567-DEF89",
  "capacityAssignmentProgress": "Completed"
}
```


C. Fabric Capacity Consumption Telemetry

- **Source**: Microsoft Fabric Capacity Metrics App.
- **Data Agent Operation**: `Operation: "AI Query"`, `Item Kind: "LlmPlugin"`.
- **Output Field**: Capacity Units (`CU Seconds`) billed against Fabric capacity.

---

2. Enterprise Fixed-Seat Subscriptions (GitHub Copilot & M365)

Enterprises license developer and office AI primarily through fixed seat licenses:

Subscription Tier	Enterprise List Price*	Telemetry API Surface	Core Raw Telemetry Extracted
GitHub Copilot Business	**$19 / user / mo**	GitHub REST API (`/billing/seats` & `/metrics/reports/ ...`)	Seat assignments, last active editor, code generation events, code acceptance events.
GitHub Copilot Enterprise	**$39 / user / mo**	GitHub REST API (`/enterprises/{ent}/copilot/ ...`)	Everything in Business + PR summaries, organization-wide knowledge base indexing.
Microsoft 365 Copilot	**$30 / user / mo**	Microsoft Graph Reports API (`/copilot/reports/ ...`)	Active vs. Inactive users, application breakdown (Teams, Word, Excel, Outlook), last active dates.
Microsoft Copilot Studio	**$200 / tenant / mo** *(25k msgs)*	Dataverse OData Web API (`conversationtranscripts`)	Total sessions, raw activity JSON, topic triggers, participant metadata.

**Prices reflect standard US commercial list pricing. Actual enterprise billing is governed by enterprise agreements (EA) and volume discounts.*

GitHub Copilot: Exact Raw API Payloads

1. Copilot Seats (Billing) API

- **Endpoint**: `GET https://api.github.com/orgs/{org}/copilot/billing/seats`
- **Raw JSON Response**:

```
{
  "total_seats": 25,
  "seats": [
    {
      "created_at": "2026-01-15T09:00:00Z",
      "updated_at": "2026-01-15T09:00:00Z",
      "pending_cancellation_date": null,
      "last_activity_at": "2026-10-24T16:15:30Z",
      "last_activity_editor": "vscode/1.94.0/copilot/1.240.0",
      "plan_type": "business",
      "assignee": {
        "login": "octocat-dev",
        "id": 583231,
        "type": "User",
        "site_admin": false
      }
    }
  ]
}
```


2. Copilot Usage Metrics Reports API

- **Endpoint**: `GET https://api.github.com/orgs/{org}/copilot/metrics/reports/users-28-day/latest`
- **Initial REST Response**:


```
{
  "report_start_day": "2026-09-27",
  "report_end_day": "2026-10-24",
  "download_links": [
    "https://copilot-metrics-storage.github.com/reports/org-users-28-day-part1.ndjson?sig=...&quot;
  ]
}
```


- **Raw NDJSON Record (Inside Downloaded Report File)**:

```
{
  "day": "2026-10-24",
  "user_login": "octocat-dev",
  "user_id": 583231,
  "code_generation_activity_count": 142,
  "code_acceptance_activity_count": 48,
  "loc_suggested_to_add_sum": 380,
  "loc_added_sum": 295,
  "totals_by_feature": [
    {
      "feature": "code_completion",
      "code_generation_activity_count": 120,
      "code_acceptance_activity_count": 45,
      "loc_suggested_to_add_sum": 380,
      "loc_added_sum": 295
    },
    {
      "feature": "copilot_app",
      "code_generation_activity_count": 22,
      "code_acceptance_activity_count": 3,
      "loc_suggested_to_add_sum": 0,
      "loc_added_sum": 0
    }
  ],
  "totals_by_language_feature": [
    {
      "language": "python",
      "feature": "code_completion",
      "code_generation_activity_count": 85,
      "code_acceptance_activity_count": 32
    }
  ]
}
```


3. Practical Implementation Strategy With Our Current Resources

3.1 Fixed Seats Telemetry: What We Need From Balaji (Admin)

To ingest authentic employee usage data without developing complex connectors, company administrators can provide access via either of two paths:

- **Option A: Microsoft 365 Copilot Usage CSV Export**
- Admin Center Path: `admin.microsoft.com → Reports → Usage → Microsoft 365 Copilot → Export`
- Delivers the direct CSV export containing all user principal names and application activity dates (`getMicrosoft365CopilotUsageUserDetail`).

- **Option B: Azure Entra App Registration Credentials**
- Parameters: `Tenant ID`, `Client ID`, `Client Secret`.
- Required Permission: Application permission `Reports.Read.All` (requires Global Admin Consent).

3.2 Type 3 Custom Autonomous Agents: Live Demonstration via Azure AI Foundry

To demonstrate real-time telemetry from a custom agent, we are building an **IT Support & Helpdesk Agent** using Azure AI Foundry tools:

- **What Agent We Are Building**:
- An enterprise IT Helpdesk Agent that handles real employee workplace requests (such as hardware troubleshooting, device boot issues, and software access).
- **Exact Output Metrics We Are Showing**:
- **Employee Identity**: Authenticated user submitting the query (`user_email`).
- **Task Category**: Classified IT request topic (e.g., `Hardware_Triage`, `Access_Provisioning`).
- **Token Consumption**: Exact `prompt_tokens` and `completion_tokens` returned directly by the model API response.
- **Execution Latency**: End-to-end response duration in milliseconds (`latency_ms`).
- **Tool Execution Status**: Success or failure status of external tool or system integrations.

---

4. How to Identify What Type of Agents Are in Your Stakeholder's GitHub Repo

When your stakeholder shares a repository containing AI agents, inspect its file tree to immediately categorize its architecture:

```
Stakeholder GitHub Repository
│
├── CASE A: Declarative Agent (M365 Copilot Extension)
│   ├── appPackage/
│   │   ├── manifest.json
│   │   └── declarativeAgent.json   <-- Look for this manifest file!
│   └── (No executable backend code; configuration package only)
│
├── CASE B: Copilot Studio Solution (Power Platform Unpack)
│   ├── Solution.xml
│   ├── bots/                       <-- Look for bots & botcomponents folders!
│   └── botcomponents/
│   └── (Exported low-code solution; runs in Dataverse)
│
└── CASE C: Code-First Custom Agent (Foundry / Semantic Kernel)
    ├── requirements.txt / pyproject.toml
    ├── semantic_kernel/ or azure.ai
    ├── plugins/
    └── main.py / agent.py          <-- Full Python/C# execution logic!
```

Action Steps Based on Repo Structure:

1. **If Case A (Declarative Agent)**:
2. Do not run the repo code. It is an M365 manifest package.
3. Ingest usage data via the **Microsoft Graph Reports API** (`/v1.0/copilot/reports/ ...`).
4. **If Case B (Copilot Studio Export)**:
5. It is a Power Platform solution.
6. Ingest session logs via the **Dataverse Web API** (`conversationtranscripts`).
7. **If Case C (Code-First Custom Agent)**:
8. It is custom software running on Azure.
9. Ingest telemetry via **Azure Application Insights** or configure a webhook in their agent event loop.
